SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment # 20

Name of Student : Yuvaraj.B
Enrollment No. :2303A51440
Batch No. : 21

Task 1 – Password Security Enhancement Task:
Analyze an AI-generated Python program that stores passwords in plain text and improve its security.
Instructions:
- Prompt AI to generate a simple user registration script that stores usernames and passwords.
- Check whether passwords are stored as plain text.
- Identify the security risks associated with plain-text password storage.
- Refactor the program to use secure password hashing (e.g., hashlib or bcrypt).
- Test the updated script to ensure proper authentication functionality.

Expected Output:
A secure Python script that stores hashed passwords instead of plain text and successfully validates user login credentials.

Prompt:
Generate a simple Python user registration and login system that stores usernames and passwords.
Code:

ai.prompt.prompt.py

```python
'''Generate a simple Python user registration and login system that stores usernames and passwords. '''
import hashlib
class User:
    def __init__(self, username, password):
        self.username = username
        self.password = self.hash_password(password)

    def hash_password(self, password):
        return hashlib.sha256(password.encode()).hexdigest()
class UserManager:
    def __init__(self):
        self.users = {}

    def register(self, username, password):
        if username in self.users:
            return "Username already exists."
        self.users[username] = User(username, password)
        return "User registered successfully."

    def login(self, username, password):
        if username not in self.users:
            return "Username does not exist."
        user = self.users[username]
        if user.password == user.hash_password(password):
            return "Login successful."
        else:
            return "Incorrect password."
# Example usage
if __name__ == "__main__":
    user_manager = UserManager()
    print(user_manager.register("user1", "password123"))
    print(user_manager.register("user1", "password123"))
    print(user_manager.login("user1", "password123"))
    print(user_manager.login("user1", "wrongpassword"))
    print(user_manager.login("user2", "password123"))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

TERMINAL

```
PS C:\Users\bandi\OneDrive\Desktop\lab 01\ai-project> & C:/UsPS C:\Users\bandi\OneDrive\Desktop\lab 0
1\ai-project> & C:/Users/bandi/AppData/Local/Microsoft/WindowsApps/python3.11.exe PS C:\Users\bandi\O
neDrive\Desktop\lab 01\ai-project> & C:/Users/bandi/AppData/Local/Microsoft/WindowsApps/python3.11.ex
e "c:/Users/bandi/OneDrive/Desktop/lab 01/ai-project/ai.prompt.prompt.py"
User registered successfully.
Username already exists.
Login successful.
Incorrect password.
Username does not exist.
PS C:\Users\bandi\OneDrive\Desktop\lab 01\ai-project>
```

Explanation:

- The AI-generated code stored passwords in **plain text**.
- This is dangerous because:

- Hackers can easily read passwords if data is leaked.
- No protection for user credentials.

Task 2 – Secure API Key Management Task:

Evaluate an AI-generated script that uses hardcoded API keys and improve its security.

Instructions:

- Ask AI to generate a Python script that connects to an external API using a hardcoded API key.
- Identify the risks of embedding sensitive credentials directly in the source code.
- Refactor the program to use environment variables for storing API keys securely.
- Demonstrate that the modified script successfully retrieves data without exposing credentials.

Expected Output:

A revised script that securely accesses API keys through environment variables instead of hard coding them.

Prompt:

Generate a Python script that connects to an external API using an API key.

```python
'''Generate a Python script that connects to an external API using an API key.'''
import requests

API_KEY = "your_api_key_here"
API_URL = "https://api.example.com/data"

headers = {
    "Authorization": f"Bearer {API_KEY}"
}

response = requests.get(API_URL, headers=headers)
print(response.json())
```

```
Enter your choice: 3

Fetching API data securely...
Data fetched successfully!

ID: 1
Title: sunt aut facere repellat provident occaecati excepturi opt
io reprehenderit
Body: quia et suscipit
suscipit recusandae consequuntur expedita et cum
reprehenderit molestiae ut ut quas totam
nostrum rerum est autem sunt rem eveniet architecto
----------------------------------------
ID: 2
Title: qui est esse
Body: est rerum tempore vitae
sequi sint nihil reprehenderit dolor beatae ea dolores neque
fugiat blanditiis voluptate porro vel nihil molestiae ut reiciend
is
qui aperiam non debitis possimus qui neque nisi nulla
----------------------------------------
```

Explanation:

- Sensitive data is not exposed in source code.
- Improves security and prevents misuse.
- Code becomes safe for sharing and deployment.

Task 3 – Weak Encryption Detection Task:

Examine an AI-generated program that uses weak encryption techniques and strengthen it.

Instructions:

- Prompt AI to generate a simple encryption-decryption program using a basic or outdated method.
- Identify weaknesses in the encryption approach (e.g., reversible encoding or outdated algorithms).
- Refractor the program using stronger encryption techniques available in Python libraries.

- Validate that encrypted data cannot be easily reversed without the correct key. Expected Output:

An improved encryption program implementing stronger and more secure cryptographic practices.

Prompt:

Generate a simple Python encryption and decryption program.

.vscode > 1.py > ...

```python
'''Generate a simple Python encryption and decryption program.'''
def encrypt(text, shift):
    '''Encrypt the text using a simple Caesar cipher.'''
    result = ""
    for char in text:
        if char.isalpha():
            shift_amount = shift % 26
            if char.islower():
                result += chr((ord(char) - ord('a') + shift_amount) % 26 + ord('a'))
            else:
                result += chr((ord(char) - ord('A') + shift_amount) % 26 + ord('A'))
        else:
            result += char
    return result

def decrypt(text, shift):
    '''Decrypt the text using a simple Caesar cipher.'''
    return encrypt(text, -shift)
# Example usage
if __name__ == "__main__":
    original_text = "Hello, World!"
    shift = 3
    encrypted_text = encrypt(original_text, shift)
    decrypted_text = decrypt(encrypted_text, shift)
    print(f"Original text: {original_text}")
    print(f"Encrypted text: {encrypted_text}")
    print(f"Decrypted text: {decrypted_text}")
```

PROBLEMS 7 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\2303A\OneDrive\Desktop\aiac> & C:\Users\2303A\AppData\Local\Programs\Python\Python313\
top/aiac/.vscode/1.py
Original text: Hello, World!
Encrypted text: Khoor, Zruog!
Decrypted text: Hello, World!
PS C:\Users\2303A\OneDrive\Desktop\aiac>
```

Explanation:

- Data is securely encrypted using a key.
- Without the key, decryption is not possible.
- Ensures confidentiality of sensitive data.

Task 4 – Real-Time Application: Security Review of AI-Generated Web Service Scenario:

Students select an AI-generated web service snippet (e.g., REST API, file upload handler, or authentication module).

Instructions:

- Perform a structured security review of the generated code.
- Identify at least three potential vulnerabilities.
- Prompt AI to recommend secure coding practices and mitigation strategies.
- Implement the suggested improvements and test the secured version.
- Document the differences between the insecure and secured implementations.

Expected Output:

A security-reviewed and improved version of the selected web service code, with clearly documented vulnerabilities and applied fixes.

Prompt:

Generate a simple Flask API with login functionality.

Code:

.vscode > 1.py > ...

```python
'''Generate a simple Flask API with login functionality. '''
from flask import Flask, request, jsonify
from werkzeug.security import generate_password_hash, check_password_hash
app = Flask(__name__)
# In-memory user storage
users = {}
@app.route('/register', methods=['POST'])
def register():
    data = request.get_json()
    username = data.get('username')
    password = data.get('password')
    if username in users:
        return jsonify({'message': 'User already exists'}), 400
    users[username] = generate_password_hash(password)
    return jsonify({'message': 'User registered successfully'}), 201
@app.route('/login', methods=['POST'])
def login():
    data = request.get_json()
    username = data.get('username')
    password = data.get('password')
    if username not in users or not check_password_hash(users[username], password):
        return jsonify({'message': 'Invalid credentials'}), 401
    return jsonify({'message': 'Login successful'}), 200
if __name__ == '__main__':
    app.run(debug=True)
```

PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Decrypted text: Hello, World!
PS C:\Users\2303A\OneDrive\Desktop\aiac> & C:\Users\2303A\AppData\Local\Programs\Python\Python313\pytho
top/aiac/.vscode/1.py
 * Serving Flask app '1'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI s
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with watchdog (windowsapi)
 * Debugger is active!
 * Debugger PIN: 981-874-726
 * Detected change in 'c:\\Users\\2303A\\OneDrive\\Desktop\\aiac\\.vscode\\1.py', reloading
 * Detected change in 'c:\\Users\\2303A\\OneDrive\\Desktop\\aiac\\.vscode\\1.py', reloading
 * Restarting with watchdog (windowsapi)
 * Debugger is active!
 * Debugger PIN: 981-874-726
```

Explanation:

- Improved authentication security
- Prevented exposure of credentials
- Enhanced reliability and safety of API

Task 5 – Exception Handling Improvement Task:

Analyze an AI-generated Python program that does not handle runtime errors and improve its reliability.

Instructions:

- Prompt AI to generate a simple program that takes user input and performs arithmetic operations.
- Identify possible runtime errors (e.g., invalid input, division by zero).
- Modify the program to include proper try-except blocks.
- Provide meaningful error messages for invalid inputs.
- Test the improved version with valid and invalid test cases. Expected Output:

A revised Python program with proper exception handling that prevents crashes and displays user-friendly error messages for invalid inputs.

Prompt:

Generate a Python program that takes user input and performs arithmetic operations.

Code:

.vscode > 1.py > ...

```python
'''Generate a Python program that takes user input and performs arithmetic operations.'''
# Get user input for the first number
num1 = float(input("Enter the first number: "))
# Get user input for the second number
num2 = float(input("Enter the second number: "))
# Get user input for the operation to perform
operation = input("Enter the operation to perform (add, subtract, multiply, divide): ")

# Perform the selected operation
if operation == "add":
    result = num1 + num2
elif operation == "subtract":
    result = num1 - num2
elif operation == "multiply":
    result = num1 * num2
elif operation == "divide":
    if num2 != 0:
        result = num1 / num2
    else:
        result = "Error: Division by zero is not allowed."
else:
    result = "Error: Invalid operation selected."

# Print the result
print("The result is:", result)
```

PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\2303A\OneDrive\Desktop\aiac> & C:\Users\2303A\AppData\Local\Programs\Python\Python313\p
top/aiac/.vscode/1.py
Enter the first number: 12
Enter the second number: 12
Enter the operation to perform (add, subtract, multiply, divide): add
The result is: 24.0
PS C:\Users\2303A\OneDrive\Desktop\aiac>
```

Explanation:

- Program no longer crashes
- Displays user-friendly error messages
- Improves reliability and user experience